

오늘 배울 내용

❖ 원형 연결 리스트

❖ 이중 연결 리스트

원형 연결 리스트 이해

❖ ‘원형 연결 리스트’란

◆ 마지막 노드의 링크가 첫 번째 노드(head node)를 가리키는 리스트

❖ 장점

◆ 한 노드에서 다른 모든 노드로의 접근이 가능

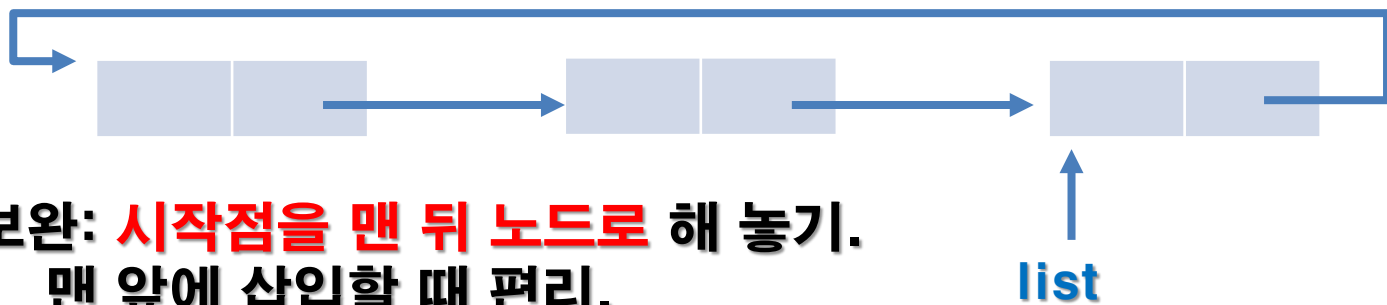
❖ 주의

◆ 검색할 때 무한루프에 빠지지 않도록 검색종료 시킬 노드를 정해야 함.

원형 연결 리스트 구조



단점: 맨 앞에 삽입하려면, 맨 뒤 원소까지 찾아가야 함.



보완: 시작점을 맨 뒤 노드로 해 놓기.
맨 앞에 삽입할 때 편리.

원형리스트 길이를 계산하는 함수 예

주의 : 검색할 때 무한루프에 빠지지 않도록
검색종료 시킬 노드를 정해야 함.

```
int length(list_pointer ptr)
{
    list_pointer temp;
    int count = 0;

    if (ptr) {
        temp = ptr;
        do {
            count++;
            temp = temp->link;
        } while (temp != ptr);
    }
    return count;
}
```

원형 연결리스트 활용 프로그래밍 연습(1/4)

❖ 원형연결리스트 구현을 위한 자기참조구조체를 정의해 보자.

◆ 구조체 이름 : **cnode**

◆ 구조체 포인터 형 선언 typedef 사용)할 때의 이름 : **npointer**

◆ 구조체 내용 : 아이디(**num**, **int**), 학점(**grade**, **char**), 연결(**link**)

◆ 시작주소로서 “**ptr**” 변수를 만들자.

```
#include <stdio.h>
#include <malloc.h>
#include <string.h>

typedef struct cnode *npointer;
struct cnode {
    int num;
    char grade;
    npointer link;
};
npointer ptr = NULL;
```

원형 연결리스트 활용 프로그래밍 연습(2/4)

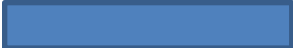
[insert_front() 함수]



```
void insert_front(npointer node)
{
    if (!ptr) {
        node -> link =         ;
        ptr =         ;
    }
    else {
        node -> link =                 ;
        ptr -> link =         ;
    }
}
```

원형 연결리스트 활용 프로그래밍 연습(3/4)

[c_print() 함수]

```
void c_print()
{
    npointer temp = ptr;
    if (ptr) {
        do {
            
        } while (  );
    }
}
```

원형 연결리스트 활용 프로그래밍 연습(4/4)

[how_many()함수]

```
int how_many()
{
    npointer temp;
    int count = 0;
    if (ptr이있다면)
    {
        temp = ptr;
        do
        {
            count++;
            temp = temp->link;
        } while (ptr과 temp가 같지 않으니 비교.)
    }
}
```


원형 연결리스트 활용 프로그래밍 연습(4/5)

[main() 함수]

```
void main()
{
    int cond=1;
    npointer temp;

    while (cond) {
        temp에 저장장소 할당.
        printf("Enter id and grade : ");
        fflush( stdin );
        scanf("%d %c", &(temp->num), &(temp->grade));
        temp를 리스트 맨 앞에 추가.
        printf("Continue?(0/1) : ");
        fflush( stdin );
        scanf("%d", &cond);
    }
    리스트 내용 프린트.
    리스트 노드수 출력.
}
```

이중 연결 리스트 이해

❖ 단순연결리스트의 문제점

- ◆ 선행 노드를 찾기 힘들다.
- ◆ cf> 삽입이나 삭제시에 반드시 선행노드가 필요.

❖ ‘이중 연결 리스트’ 란

- ◆ 하나의 노드가 **선행 노드**와 **후속 노드**에 대한 **두 개의 링크**를 가지는 리스트
- ◆ 원형구조일 수도 있고 아닐 수도 있음.

❖ 장점

- ◆ 링크가 양방향이므로 **양방향으로 검색이 가능**

이중 연결 리스트의 노드 구조 정의

- ❖ 이중연결리스트 구현을 위한 자기참조구조체를 정의해 보자.
 - ◆ 구조체 이름 : **node**
 - ◆ 구조체 포인터 형 선언(**typedef** 사용)할 때의 이름 : **node_pointer**
 - ◆ 구조체 내용 : 왼쪽연결(**llink**), 데이터(**id, grade**), 오른쪽연결(**rlink**)

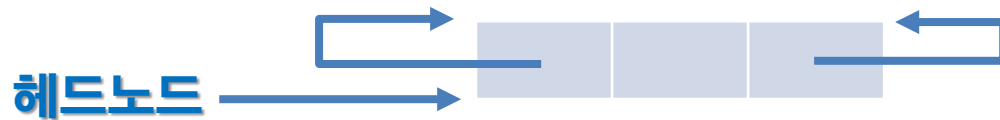
```
typedef struct node *node_pointer;

struct node {
    node_pointer llink; // left node
    int id;             //data
    char grade;
    node_pointer rlink; // right node
};
```

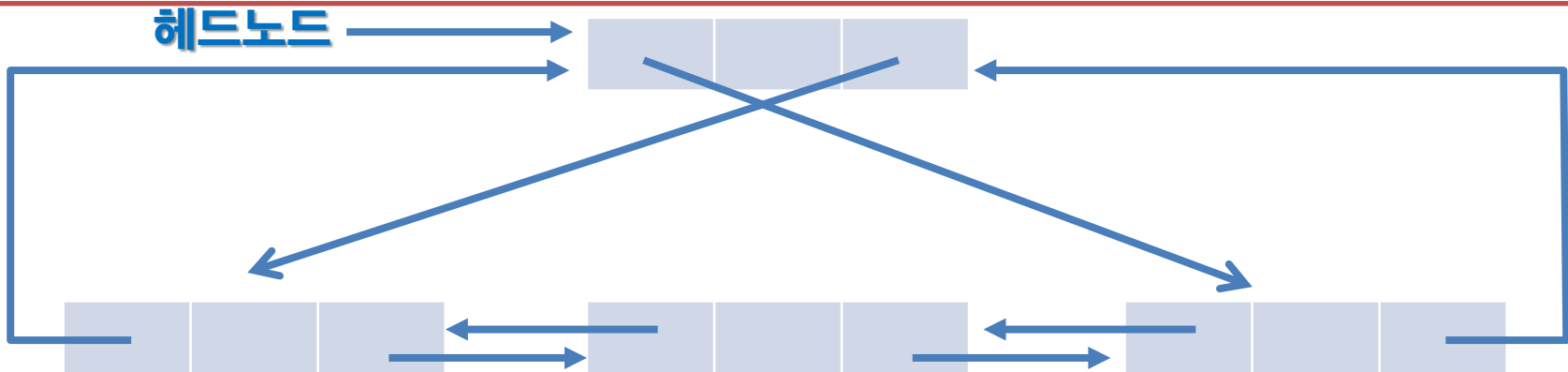
이중 연결 리스트 구조

❖ 헤드 노드를 사용하여 연산을 쉽게 함

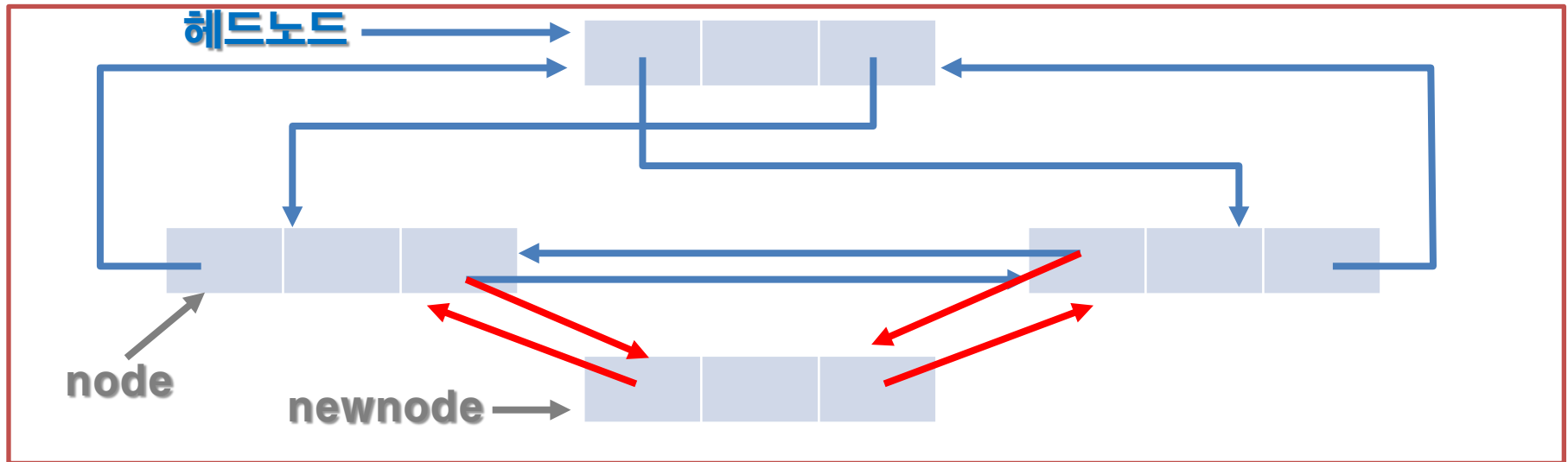
◆ **헤드노드** : 아무 정보 없이 연산을 돕기 위해 존재.
맨 앞 노드보다 앞에, 맨 뒤 노드보다 뒤에 있다고 생각.



헤드 노드를 가진 공백 이중 연결 리스트

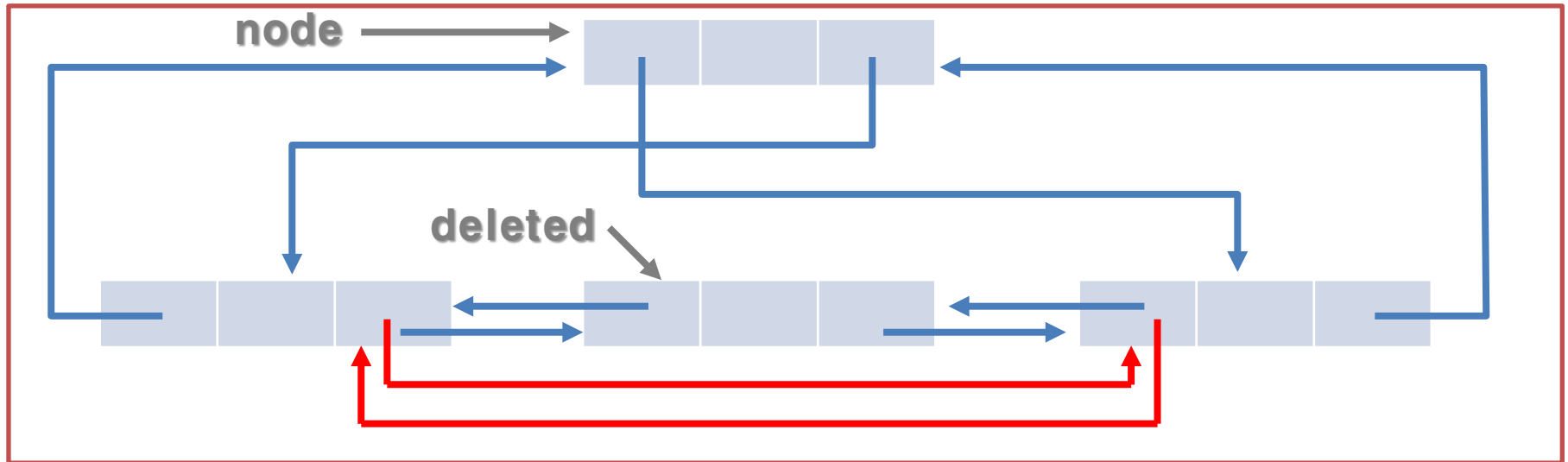


헤드 노드를 가진 이중 연결 원형 리스트



```
void dinsert(node_pointer node, node_pointer newnode)
{
    /* newnode를 node의 오른쪽에 삽입 */
    newnode의 llink, rlink 정하기
    node이용하여 앞,뒤 노드의 link 정하기
}
```

이중연결 리스트로부터 노드 삭제



```
void ddelete(node_pointer node, node_pointer deleted)
{
    /* head node가 "node" 인 이중 연결 리스트에서 삭제 */
    if ( )
        printf("Deletion of head node not permitted.\n");
    else {
        deleted의 앞,뒤 노드의 link 수정하기
        free(deleted);
    }
}
```